

E-Sports Tournament & Competitive Gaming Platform

TAE-2 Database Implementation & Advanced SQL

TY B.Tech – Data Science • G H Raisoni College of Engineering and Management, Pune

Dhananjay Singh [P13]

RDBMS: MySQL (Workbench)

PLAYER

VIP_PASS

TEAM_ROSTER

GAME_TITLE

COMPETITIVE_LEAGUE

MATCH_SCHEDULE

TOURNAMENT_ENTRY

PRIZE_PAYOUT

PAYOUT_TRANSACTION

MATCH_PERFORMANCE

ANTICHEAT_VIOLATION

11 normalised relations • 3NF / BCNF

1. Domain & TAE-1 Recall

Competitive Gaming & Esports Operations

- Manages players, VIP battle passes, game titles, leagues, teams, match fixtures, entries, prize payouts, match telemetry and anti-cheat flags.
- TAE-1: conceptual design → EER model and 11 relations normalised to 3NF / BCNF (approved).
- TAE-2: implement the schema in MySQL Workbench; demonstrate advanced SQL, database objects and tuning.
- Raw source: 1,250-record, 41-attribute messy dataset, cleaned into the 11 tables.

Player lifecycle

- 1 Player Registration & VIP Pass
- 2 Team Roster Join
- 3 Tournament Entry
- 4 Match Fixture Execution
- 5 Prize Payout & Performance Audit

2. Database Schema

Eleven Core Tables, Relationships and Integrity Rules

Players & Teams

- PLAYER
- VIP_PASS
- TEAM_ROSTER

Competition

- GAME_TITLE
- COMPETITIVE_LEAGUE
- MATCH_SCHEDULE
- TOURNAMENT_ENTRY

Payments

- PRIZE_PAYOUT
- PAYOUT_TRANSACTION

Telemetry & Integrity

- MATCH_PERFORMANCE
- ANTICHEAT_VIOLATION

Relationships

- GAME_TITLE 1:N LEAGUE 1:N MATCH_SCHEDULE
- PLAYER 1:1 VIP_PASS (UNIQUE foreign key)
- TOURNAMENT_ENTRY resolves Player-Match M:N
- ENTRY 1:1 PRIZE_PAYOUT 1:1 PAYOUT_TRANSACTION
- MATCH_PERFORMANCE 1:N ANTICHEAT_VIOLATION

Constraints (DDL)

- PRIMARY KEY / FOREIGN KEY on all 11 tables
- UNIQUE: IGN, Email, Team_Tag, Txn_Ref_Hash
- CHECK: MMR ≥ 0 , Prize ≥ 0 , Headshot_Pct 0-100
- CHECK: Team_A_ID $\neq$ Team_B_ID in a match
- NOT NULL + CHECK domains for status columns

3. Advanced SQL Queries

Joins, Correlated Subqueries and Aggregation

Multi-table Joins

INNER · LEFT · SELF

INNER: PLAYER →
TOURNAMENT_ENTRY →
MATCH_SCHEDULE → LEAGUE →
GAME_TITLE for entries per game

LEFT: PLAYER LEFT JOIN
MATCH_PERFORMANCE to find
players who never played

SELF: PLAYER self-join pairing same-
Country players with close MMR

Correlated Subqueries

Evaluated per outer row

Above own average: match records
where Combat_Score > that player's
average

Above group average: players
whose MMR_Rating exceeds their
country's average

EXISTS: players with an anti-cheat
flag above 90% confidence

Aggregate Queries

GROUP BY + HAVING

GROUP BY Player_ID: top players by
average KDA and combat score

GROUP BY Team_ID: win / loss ratio
per team

HAVING COUNT(*) ≥ 3: repeat anti-
cheat offenders by cheat type

4. Database Objects

Two Views, One Stored Procedure, One Trigger

VIEW

`vw_player_leaderboard`

- Ranks players by MMR with average KDA and combat score
- `PLAYER LEFT JOIN MATCH_PERFORMANCE · AVG + RANK() OVER`
- Virtual view: always reflects live data

STORED PROCEDURE

`sp_process_prize_payout(...)`

- Params: entry, placement, prize, tax, method, hash
- Inserts `PRIZE_PAYOUT` + `PAYOUT_TRANSACTION` in one transaction (keeps 1:1)
- `Amount_Paid = prize - tax`; rejects invalid input

VIEW

`vw_league_payout_summary`

- Prize pool vs. amount actually paid, per league
- `GAME_TITLE → LEAGUE → MATCH → ENTRY → PAYOUT → TXN`
- Virtual view: reporting without duplicating data

TRIGGER

`trg_flag_suspicious_performance`

- AFTER INSERT ON `MATCH_PERFORMANCE` FOR EACH ROW
- `Headshot_Pct > 90` or `KDA_Ratio > 15` → auto-insert `ANTICHEAT_VIOLATION`
- Builds an automatic anti-cheat audit trail

5. Performance Tuning & Indexing

EXPLAIN Before and After Indexing

Test 1 — Country + MMR lookup

Query filters: Country, MMR_Rating (PLAYER)

```
CREATE INDEX idx_player_country_mmr ON PLAYER  
(Country, MMR_Rating);
```

Composite index · expected plan: type ALL (full scan) → ref / range

Test 2 — High-confidence flags

Query filter: Flag_Confidence_Pct (ANTICHEAT_VIOLATION)

```
CREATE INDEX idx_violation_conf ON  
ANTICHEAT_VIOLATION (Flag_Confidence_Pct);
```

Single-column index · InnoDB already indexes foreign keys, so we index non-key filter columns

Documenting the improvement

EXPLAIN (before)

→

CREATE INDEX

→

ANALYZE TABLE

→

EXPLAIN (after)

Compare EXPLAIN columns: type · key · rows · Extra

6. Final Deliverables

Three SQL Files Submitted for TAE-2

schema.sql

Structure & objects

- CREATE TABLE for all 11 relations with PK, FK, UNIQUE, CHECK
- 2 views, 1 stored procedure (DELIMITER \$\$), 1 trigger

data.sql

Sample data

- INSERT statements loaded in FK order, parents first
- Cleaned from the 1,250-record raw dataset
- Verified with SELECT COUNT(*) per table

queries.sql

Analysis & tuning

- Joins, correlated subqueries, GROUP BY + HAVING
- View, procedure and trigger demonstrations
- EXPLAIN → CREATE INDEX → EXPLAIN

Run order

schema.sql

→

data.sql

→

queries.sql

```
mysql -u root -p esports_db < schema.sql (or run each file in Workbench)
```